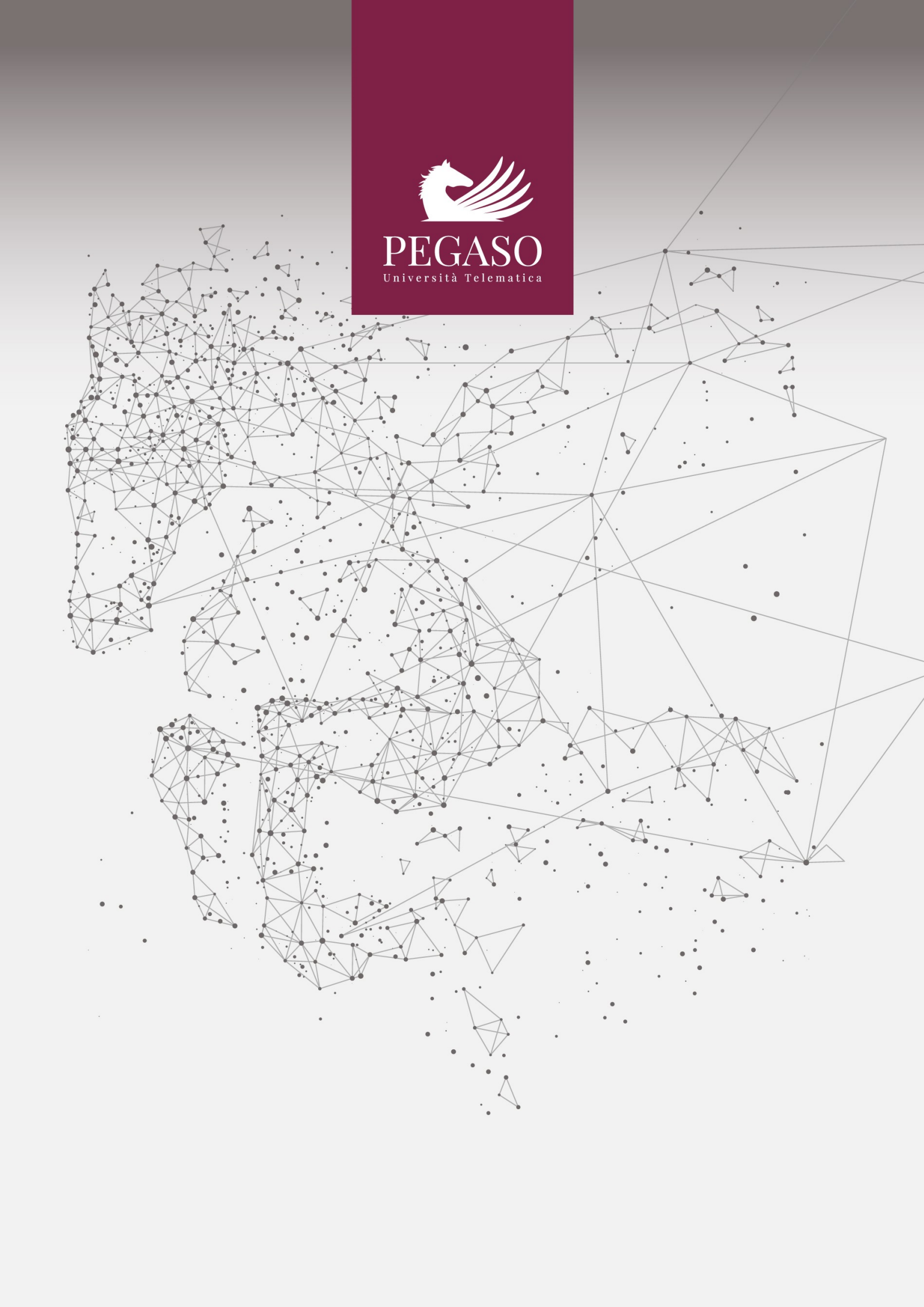

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE 3
2. CONCETTO DI CLASSE 4
2.1 DEFINIZIONE DEL COSTRUTTORE E DEI METODI DELLE CLASSI 6
3. OGGETTI COME ISTANZE DI UNA CLASSE 10
4. ESEMPIO DI PROGETTAZIONE DI UNA CLASSE 13
5. EREDITARIETÀ 18
6. POLIMORFISMO 22
BIBLIOGRAFIA 24

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In questa dispensa di accompagnamento alla video-lezione, si introduce il paradigma di programmazione ad oggetti e si presentano gli strumenti che Python mette a disposizione per l'implementazione di tale paradigma. Vengono, inoltre, illustrati i tre pilastri della programmazione ad oggetti: incapsulamento, ereditarietà e polimorfismo e presentati alcuni esempi di applicazione dei concetti stessi.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Concetto di classe

Nella programmazione, finora, abbiamo utilizzato oggetti già progettati e programmati come, ad esempio, l'oggetto file con i suoi attributi e i suoi metodi (es. write()).

In questa lezione si vedrà come progettare e programmare delle nuove **classi** (e quindi nuovi oggetti) sulla base delle necessità per la risoluzione dei problemi.

A differenza della programmazione strutturata in cui le funzioni operano sui dati (variabili) del programma, la **programmazione orientata agli oggetti** è focalizzata sulle operazioni sugli oggetti.

La programmazione orientata agli oggetti è nota anche come **OOP (Object-Oriented Programming**). La OOP risolve il problema della crescente complessità software, permette una certa facilità di comprensione e modifica del codice, oltre che robustezza e riusabilità. Inoltre, la OOP prevede che un programma corrisponda ad un'interazione tra oggetti.

I tre **pilastri** fondamentali della programmazione ad oggetti sono:

- **incapsulamento;**
- **ereditarietà;**
- **polimorfismo.**

L'incapsulamento combina dati e codice in un singolo oggetto e permette, di fatto, **l'information hiding** (letteralmente occultamento delle informazioni). I dati dell'oggetto (**attributi**) sono, infatti, nascosti verso l'esterno. Ciò permette l'accesso agli attributi solo da parte dei metodi dell'oggetto e la protezione da errori accidentali. Attraverso l'incapsulamento, inoltre, il codice esterno all'oggetto non necessita di conoscere la struttura interna dell'oggetto stesso. I concetti di ereditarietà e di polimorfismo, invece, saranno trattati nei prossimi paragrafi.

Nel paradigma OOP, ogni oggetto appartiene ad una classe di oggetti simili e ha degli attributi (variabili riferiti all'oggetto stesso). L'insieme dei valori degli attributi ne definiscono lo **stato** (per es. per l'oggetto orologio lo stato è rappresentato dall'insieme dei valori degli attributi secondi, minuti e ore). Inoltre, ogni oggetto può eseguire delle azioni (funzioni denominate **metodi**)

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

che ne modificano lo stato. L'insieme dei metodi di un oggetto prende il nome di **interfaccia** dell'oggetto. La Figura 1 mostra schematicamente il concetto di interfaccia dell'oggetto costituita dai metodi che permettono di accedere agli attributi secondo il principio dell'information hiding.

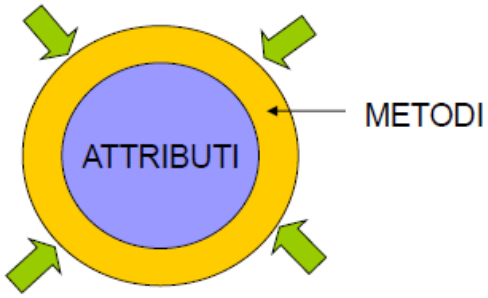


Figura 1. Schema di un oggetto: l'interfaccia (in giallo) costituita dai metodi permette l'accesso agli attributi

La classe, dunque, è il codice che specifica gli attributi e i metodi di un particolare tipo di oggetto (una specie di «stampo» che definisce forma e comportamento di un'entità). Con il termine **istanza**, invece, si definisce un oggetto creato dalla classe con le caratteristiche definite dalla classe. Per una determinata classe ci possono essere più istanze, ovvero più oggetti.

Nella classe Automobile, ad esempio, *cilindrata, velocità istantanea, marca* e *modello* saranno gli attributi della classe, mentre *accelera(), frena(), accendi()* e *spegni()* saranno i metodi della classe che, generalmente, agiscono sugli attributi dell'oggetto stesso. A titolo esemplificativo, il metodo *accelera()* andrà a modificare l'attributo velocità istantanea nel tempo.

Un'istanza (oggetto) della classe Automobile mantiene tutti i metodi e fornisce un valore per gli attributi: es. cilindrata = 3902, velocità istantanea=0, marca= «Ferrari», modello = «488 GTB».

In Python, come negli altri linguaggi di programmazione, si può definire una classe "personalizzata". Ricordando che in Python i nomi delle classi iniziano sempre con la lettera maiuscola, il formato generale di definizione della classe è il seguente:

```
class Nome_classe:
```

La definizione dei metodi di una classe avviene come quella delle altre funzioni in Python.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

L'unica differenza è la "variabile" **self** che si riferisce allo specifico oggetto su cui si sta lavorando.

2.1 Definizione del costruttore e dei metodi delle classi

Ogni classe ha un metodo di inizializzazione, denominato **costruttore**. Tale metodo viene eseguito automaticamente alla creazione di un'istanza (oggetto). Il costruttore inizializza gli attributi e definisce il parametro self dell'oggetto appena creato.

Il formato del costruttore è il seguente:

```
def __init__(self):
```

Nel codice sorgente di esempio che segue si dichiara la classe Moneta. L'attributo della classe Moneta sarà *latosu* che identifica quale lato della moneta è rivolto verso l'alto.

L'inizializzazione dell'oggetto (quando verrà istanziato) vedrà la moneta con il lato *Testa* verso l'alto.

```
class Moneta:

    def __init__(self):
        self.latosu = "Testa"
```

Si definiscono, ora, gli altri metodi della classe Moneta:

- il metodo *lancia()* che simulerà il lancio della moneta;
- il metodo *get_latosu()* che restituirà il valore attuale del lato della moneta rivolto verso l'alto.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
class Moneta:

    def __init__(self):
        self.latosu = "Testa"

    def lancia(self):
        if random.randint(0,1) == 0:
            self.latosu = "Testa"
        else:
            self.latosu = "Croce"


    def get_latosu(self):
        return self.latosu
```

Oltre al costruttore, ci sono alcuni metodi standard per gli oggetti. Uno di questi è il metodo ***__str__()*** che mostra lo stato dell'oggetto. Esso viene chiamato automaticamente quando l'oggetto viene passato come argomento alla funzione *print()* o alla funzione *str()*. Di seguito si riporta la definizione del metodo *__str__()* all'interno della classe Moneta:

```
def __str__(self):
    return "Dopo l'ultimo lancio è uscito: "+self.latosu
```

Il metodo *__str__()* attraverso le seguenti istruzioni nel main viene richiamato con il metodo *print()* (l'attributo latosu sarà uguale a testa perché è l'oggetto *m* è stato solo inizializzato attraverso il costruttore):

```
def main():
    m = Moneta()

    print(m)
```

Si riporta la visualizzazione a schermo dell'elaborazione del codice precedente:

```
Dopo l'ultimo lancio è uscito: Testa
```

Al fine di permettere l'accesso agli attributi attraverso i soli metodi dell'oggetto (interfaccia), gli attributi dovrebbero essere definiti tutti come privati. Per definire gli attributi privati, è necessario anteporre al nome dell'attributi due trattini bassi __ (es. __latosu). Il codice seguente,

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

riporta il costruttore della classe moneta indicando, però, l'attributo *latosu* come privato (in tutti i metodi l'attributo privato va chiamato anteponendo i due trattini bassi):

```
def __init__(self):
    self.__latosu = "Testa"
```

Se gli attributi non fossero tutti privati, nel semplice esempio della moneta con attributo *latosu* non privato, sarebbe possibile barare nel codice del main nel seguente modo:

```
def main():
    m = Moneta()

    print(m.get_latosu())
    m.latosu = "Croce"
    print(m.get_latosu())
```

Il lancio della moneta, infatti, non sarebbe casuale, ma predeterminato dal codice della funzione main.

Nel momento in cui tutti gli attributi della classe sono privati, devono essere forniti dei metodi per accedere ad essi e modificarli. Tali metodi prendono i nomi di **getter** e **setter**. I metodi getter (es. get_saldo()) restituiscono il valore dell'attributo della classe senza modificarlo. Ciò rappresenta un modo sicuro per recuperare il valore degli attributi da parte di codice esterno alla classe.

I metodi setter (es. set_colore()), invece, memorizzano o modificano il valore di un attributo effettuando, se necessario, anche controlli e/o elaborazioni.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Per rappresentare una classe, si utilizza il **diagramma UML delle classi (UML sta per Unified Modelling Language)**. Il layout del diagramma è composto da tre parti: nome classe, lista attributi, lista dei metodi, come indicato in Figura 2.

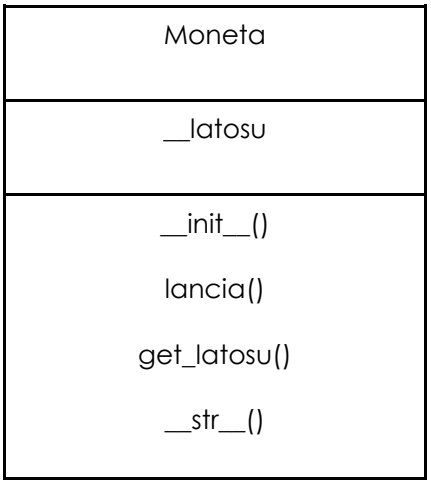


Figura 2. Diagramma UML della classe di esempio Moneta

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Oggetti come istanze di una classe

Per creare una nuova istanza della classe (**oggetto**), andrà richiamato in una funzione (es. main) il costruttore della classe stessa. Per chiamare i metodi dell'istanza si usa la dot notation (si noti che il riferimento a self viene passato automaticamente). Il codice seguente definisce la funzione main in cui viene istanziato l'oggetto *m* di tipo Moneta e viene visualizzato a schermo il lato della moneta rivolto verso l'alto. Successivamente, si effettua la simulazione di 5 lanci e, per ognuno di questi, la stampa del lato della moneta rivolto verso l'alto.

```
def main():
    m = Moneta()

    print(m.get_latosu())
    for i in range(5):
        m.lancia()
        print(m.get_latosu())
```

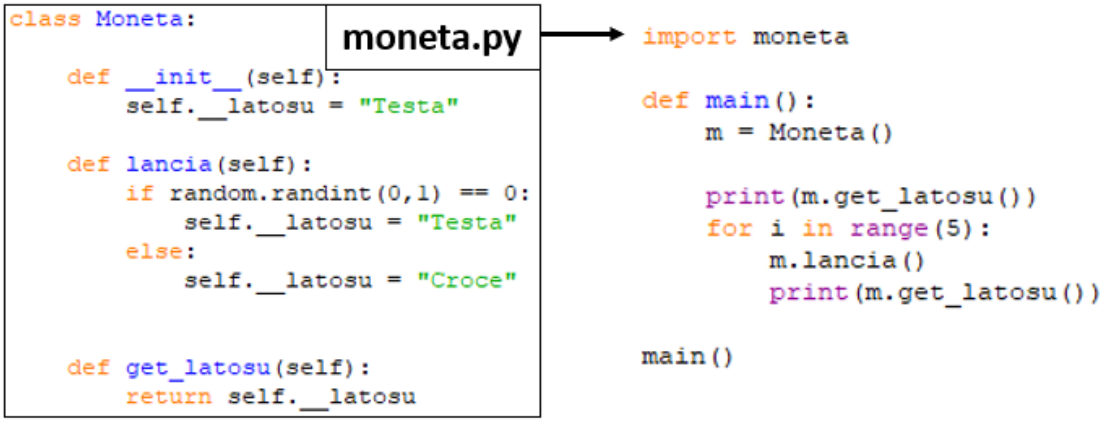
Il risultato dell'elaborazione sarà il seguente (si sottolinea che ogni esecuzione del main sarà diversa dall'altra perché i lanci saranno casuali):

```
Testa
Croce
Croce
Testa
Croce
Testa
```

Al fine di una migliore leggibilità e riusabilità del codice, le classi possono essere incluse in **moduli**. Il nome del file del modulo dovrà avere estensione .py (nell'esempio moneta.py). Il modulo può essere, quindi, importato dai programmi che usano la classe.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).



In Python, così come con altri linguaggi di programmazione, è possibile lavorare con più istanze della stessa classe. Ogni istanza/oggetto avrà il suo stato, ovvero il suo insieme di valori degli attributi. Nel codice sorgente di esempio che segue, vengono istanziate due monete e ciascuna di esse viene lanciata per 5 volte. Successivamente, vengono stampati i valori delle due monete:

```
def main():
    m1 = Moneta()
    m2 = Moneta()

    for i in range(5):
        m1.lancia()
        m2.lancia()

    print("La moneta 1 è: ",m1," - la moneta 2 è: ",m2)

main()
```

Nel risultato dell'elaborazione che segue, si noti come, volutamente, è stato richiamato il metodo standard *__str__()* dalla funzione *print()*. È, infatti, molto importante scegliere la corretta forma della stringa restituita dal metodo *__str__()* per evitare che l'output di un oggetto sia visualizzato a schermo in un modo non propriamente corretto come il seguente:

```
La moneta 1 è:  Dopo l'ultimo lancio è uscito: Croce
- la moneta 2 è:  Dopo l'ultimo lancio è uscito: Testa
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

I metodi e le funzioni accettano spesso gli **oggetti come argomenti**. Quando si passa un oggetto come argomento, in realtà si passa il riferimento all'oggetto. Il metodo o la funzione "ricevente" accedono all'oggetto:

- i metodi degli oggetti possono essere chiamati nella funzione o nel metodo "ricevente";
- gli attributi possono essere modificati utilizzando i metodi setter/getter.

Per scopi didattici si propone la funzione *lancia_moneta()* a cui si passa un oggetto di tipo moneta. La funzione *lancia_moneta()* richiamerà il metodo *lancia()* dell'oggetto moneta:

```
def lancia_moneta(m):
    m.lancia()


def main():

    m1 = Moneta()
    lancia_moneta(m1)
    print(m1)
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Esempio di progettazione di una classe

Verrà ora presentato un esempio di modellazione di una classe in Python. La classe selezionata è *Conto_corrente*, una classe che permette la simulazione della gestione di un conto corrente.

Nella trattazione che segue, nella definizione dei metodi, si ometterà il passaggio del parametro self che di default deve essere passato al metodo.

Dopo la definizione della classe, il costruttore inizializza l'importo *__saldo* (come attributo privato) del credito iniziale del conto con il valore *saldo* che gli viene passato come argomento.

Il metodo *deposito()* simula il deposito di denaro aggiungendo al credito attuale del conto contenuto nell'attributo privato *__saldo*, il valore *importo* del deposito.

Il metodo *ritiro()* simula il prelievo di denaro dal conto. Prima di procedere con il prelievo, il metodo controlla che il saldo corrente sia maggiore o uguale all'importo da prelevare. In caso positivo procede al prelievo, mentre in caso negativo stampa il messaggio "Saldo insufficiente".

Il metodo getter *get_saldo()* restituisce il valore del saldo corrente. Il metodo setter non viene dichiarato perché, come avviene per i conto correnti reali, l'unico momento in cui al saldo può essere assegnato direttamente un valore è all'apertura del conto. Il saldo del conto corrente, infatti, cresce o si riduce per depositi o prelievi successivi.

Infine, il metodo standard *__str__()* restituisce una stringa con il valore del saldo del conto corrente.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il codice sorgente della classe appena descritta è il seguente:

```
class Conto_corrente:

    def __init__(self, saldo):
        self.__saldo = saldo


    def deposito (self, importo):
        self.__saldo += importo


    def ritiro (self, importo):
        if self.__saldo >= importo:
            self.__saldo -= importo
        else:
            print("Saldo insufficiente")


    def get_saldo(self):
        return self.__saldo


    def __str__(self):
        return "Il saldo è pari a € " + format(self.__saldo, ",.2f")
```

Nella funzione *main()* che segue, si inseriscono poche istruzioni per illustrare il funzionamento minimo del simulatore del conto corrente. In particolare, in input si richiede il valore del saldo iniziale del conto corrente (che verrà memorizzato nella variabile *saldo_partenza*).

Successivamente, si istanzierà l'oggetto *conto* di tipo Conto_corrente con il saldo iniziale passato come unico parametro (il parametro self viene, invece, sempre passato automaticamente).

In input si richiede il valore della paga da depositare sul conto e si procede al deposito di tale valore. Si stampa, dunque, il valore del saldo corrente.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Infine, si richiede in input l'importo da prelevare e si procede al prelievo. Si stampa, quindi, il valore del saldo corrente.

```
def main():

    saldo_partenza = float(input("Inserire saldo iniziale: "))
    conto = Conto_corrente(saldo_partenza)

    paga = float(input("Inserire paga mensile: "))
    conto.deposito(paga)
    print(conto)

    r = float(input("Inserire importo da prelevare: "))
    conto.ritiro(r)
    print(conto)

main()
```

Il risultato dell'elaborazione per valori di esempio degli importi di saldo iniziale, deposito e prelievo, è il seguente:

```
Inserire saldo iniziale: 1000
Inserire paga mensile: 600
Il saldo è pari a € 1,600.00
Inserire importo da prelevare: 500
Il saldo è pari a € 1,100.00
```

Nella risoluzione degli esercizi, è importante, ad esercizio terminato, prevedere anche le istruzioni per la gestione delle eccezioni. Si riporta di seguito il codice della funzione main() appena trattato con la gestione delle eccezioni (la tipologia di eccezione gestita è riportata nel codice sorgente come commento in rosso):

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
    def main():

        #input di valore numerico di saldo di partenza
        #con gestione delle eccezioni
        corretto = False
        while not corretto:
            try:
                saldo_partenza = float(input("Inserire saldo iniziale: "))
                corretto = True
            except:
                print("Errore! Valore inserito non numerico")

        #nuova istanza di conto corrente
        conto = Conto_corrente(saldo_partenza)

        #input di valore numerico della paga mensile da depositare
        #con gestione delle eccezioni
        corretto = False
        while not corretto:
            try:
                paga = float(input("Inserire paga mensile: "))
                corretto = True
            except:
                print("Errore! Valore inserito non numerico")

        conto.deposito(paga)
        print(conto)

    #input di valore numerico di importo da ritirare con gestione delle eccezioni
    corretto = False
    while not corretto:
        try:
            r = float(input("Inserire importo da prelevare: "))
            corretto = True
        except:
            print("Errore! Valore inserito non numerico")

    conto.ritiro(r)
    print(conto)


main()
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il risultato dell'elaborazione con eccezioni simulate per testare il funzionamento del codice precedente è il seguente:

```
Inserire saldo iniziale: ciao
Errore! Valore inserito non numerico
Inserire saldo iniziale: ciao
Errore! Valore inserito non numerico
Inserire saldo iniziale: 200
Inserire paga mensile: 10
Il saldo è pari a € 210.00
Inserire importo da prelevare: ora
Errore! Valore inserito non numerico
Inserire importo da prelevare:
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

5. Ereditarietà

Come anticipato nel capitolo 2, il secondo pilastro della OOP è **l'ereditarietà**. È possibile individuare e progettare con oggetti che rappresentano la **versione specializzata** di altri oggetti.

Un oggetto specializzato ha tutte le caratteristiche di quello generale più altre caratteristiche uniche, tipiche della specializzazione. Un esempio può essere quello dell'oggetto di tipo cerchio, una specializzazione dell'oggetto di tipo figura geometrica.

Come nel caso precedente, solitamente, è possibile individuare una **relazione "is a"** tra oggetto specializzato e oggetto generale.

La **superclasse** (o **classe base**) è la classe generale da cui si istanziano gli oggetti generali.

La **sottoclasse** (o **classe derivata**) è la classe specializzata da cui si istanziano gli oggetti specializzati. La classe derivata è una versione estesa della classe base che eredita gli attributi e i metodi della classe base e ne aggiunge di nuovi.

Una volta definita completamente la superclasse (o classe base), per indicare l'ereditarietà, il nome della superclasse viene inserito tra parentesi dopo il nome della sottoclasse. Nell'esempio del cerchio e della figura geometrica, il codice Python è:

```
class Cerchio(Figura_geometrica):
```

Il costruttore di una sottoclasse chiama il costruttore della superclasse e poi inizializza gli attributi specifici della sottoclasse (**overriding del costruttore**).

Si aggiungono, poi, le definizioni dei metodi specifici della sottoclasse (i metodi ereditati dalla superclasse non vanno scritti nuovamente).

A titolo esemplificativo, si riporta il codice sorgente della superclasse Figura_geometrica. Gli attributi di tale classe sono x e y, cioè le coordinate bidimensionali della posizione della figura geometrica. I metodi sono il costruttore che inizializza i valori di x e y, i setter e i getter di x e y e il metodo *__str__()*.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
import math

class Figura_geometrica:

    def __init__(self, x, y):
        self.__x = x
        self.__y = y

    def set_x(self,x):
        self.__x = x

    def get_x(self):
        return x

    def set_y(self,y):
        self.__y = y

    def get_y(self):
        return y

    def __str__(self):
        return "("+str(self.__x)+","+str(self.__y)+")"
```

La classe derivata Cerchio, eredita gli attributi e i metodi della classe base e:

- ridefinisce il costruttore che richiama quello della classe base passandogli parte degli argomenti (Figura_geometrica.__init__(self,x,y));
- aggiunge l'attributo raggio r;
- aggiunge setter/getter del raggio r;
- aggiunge i metodi *calcola_perimetro()* e *calcola_area()*;
- ridefinisce il metodo *__str__()* richiamando, nella concatenazione delle stringhe, anche il metodo *__str__()* della superclasse.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
class Cerchio(Figura_geometrica):

    def __init__(self,x,y,r):
        Figura_geometrica.__init__(self,x,y)
        self.__r = r

    def set_r(self, r):
        self.__r = r

    def get_r(self):
        return r

    def calcola_perimetro(self):
        return 2*math.pi*self.__r

    def calcola_area(self):
        return self.__r*(math.pi**2)

    def __str__(self):
        return "Cerchio di raggio "+str(self.__r)+" cenrato in "\
               + Figura_geometrica.__str__(self)
```

Un esempio di utilizzo delle classi appena trattate è riportato nel codice seguente in cui viene inizializzato un cerchio in posizione (0,0) di raggio 1. Vengono, poi, stampati i valori di perimetro e di area del cerchio:

```
def main():
    c = Cerchio(0,0,1)
    print(c)
    print("Il perimetro del cerchio è:",format(c.calcola_perimetro(),",.2f"))
    print("L'area del cerchio è:",format(c.calcola_area(),",.2f"))
```

Il risultato dell'elaborazione è il seguente:

```
Cerchio di raggio 1 cenrato in (0,0)
Il perimetro del cerchio è: 6.28
L'area del cerchio è: 9.87
```

Per rappresentare l'ereditarietà tra classi nei diagrammi UML, si utilizza una freccia (punta a triangolo vuoto) dalla sottoclasse alla superclasse, come indicato in Figura 3.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

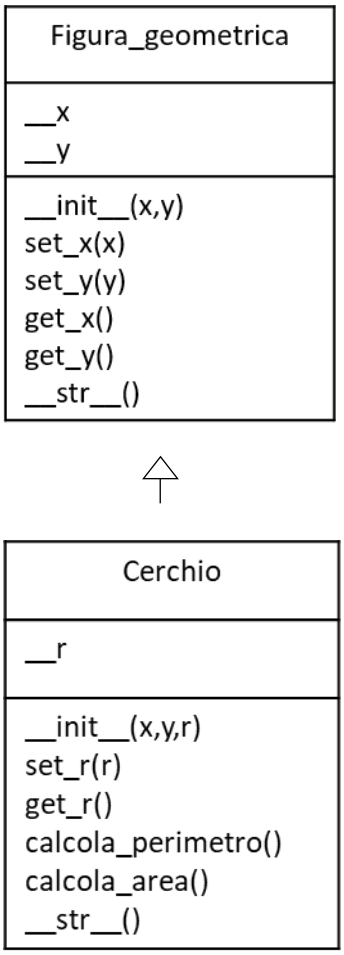


Figura 3. Diagramma UML di una superclasse e di una sottoclasse

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

6. Polimorfismo

Come anticipato nel capitolo 2, il terzo pilastro della OOP è il **polimorfismo**. Il polimorfismo è l'abilità di un oggetto ad assumere diverse forme. Gli elementi essenziali per un comportamento polimorfico sono:

- la capacità di definire un metodo nella superclasse e fare **l'override** del metodo stesso in una sottoclasse;
- la sottoclasse definisce il metodo con lo stesso nome, ma comportamento relativo a quello definito nella sottoclasse;
- la capacità di chiamare la versione corretta del metodo **overridden** che dipende dal tipo di oggetto che l'ha chiamato.

Per scopi didattici, si riporta il codice con **overriding** del metodo *calcola_area()*. Nella superclasse, tale metodo restituirà una stringa a schermo:

```
class Figura_geometrica:

    def calcola_area(self):
        return "Elementi insufficienti per il calcolo dell'area"
```

Nella sottoclasse, tale metodo restituirà il valore dell'area del cerchio calcolata in base al raggio:

```
class Cerchio(Figura_geometrica):

    def calcola_area(self):
        return self.__r*(math.pi**2)
```

Nel codice della funzione main() seguente, si istanziano due oggetti: uno figura geometrica e uno cerchio. Si richiama, in seguito, la funzione *calcola_area()* sia per l'oggetto f che per l'oggetto c.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il comportamento del metodo (istruzioni eseguite) sarà diverso in base al tipo di oggetto che ha chiamato il metodo:

```
def main():
    f = Figura_geometrica(1,1)
    c = Cerchio(0,0,1)
    print("L'area della figura geometrica è:",f.calcola_area())
    print("L'area del cerchio è:",c.calcola_area())
```

Il risultato dell'elaborazione del codice precedente è:

```
L'area della figura geometrica è: Elementi
insufficienti per il calcolo dell'area
L'area del cerchio è: 9.869604401089358
```

L'overriding differisce dal meccanismo dell'**overloading**. L'overloading è un diverso comportamento della funzione (istruzioni diverse) con stesso nome, ma con diverso numero di parametri. Si prendano, per esempio, le seguenti due funzioni:

def somma(self,a,b)

def somma(self,a,b,c)

L'overriding prevede, invece, che il nome e i parametri dei metodi siano uguali, ma il comportamento dipenda dall'effettiva classe dell'oggetto su cui il metodo è stato chiamato.

Il polimorfismo fornisce una grande flessibilità quando si progettano i programmi.

Ad ogni modo, esistono delle eccezioni che possono essere sollevate con l'overriding. Ad esempio, l'**eccezione AttributeError** viene sollevata quando un metodo riceve un oggetto che non è un'istanza della classe corretta.

Esiste, infine, la **funzione isinstance()** che determina se l'oggetto è un'istanza della classe (ritorna True o False). Il formato d'uso di tale funzione è:

isinstance(object, class)

L'istruzione seguente permette di richiamare la funzione *print()* passandogli il risultato della funzione *isinstance()* che verifica se l'oggetto c è di classe Cerchio (in questo caso True):

```
print(isinstance(c,Cerchio))
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Gaddis, T. (2016). Introduzione a Python. Ediz. mylab. Con espansione online. Pearson.
- Python (2021). https://www.python.org/

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).